directly by obtaining a reference to the entity and then sending the message with a normal `tell(!)`.

During normal rebalancing—that is, not during a failure—messages are kept in a buffer until the displaced entity is restored. However, if you lose a node, you may lose messages before all regions notice the loss and start buffering: the messages are not buffered until the SR is marked unavailable.

9.1.2 An example: A shipping container

Let's say you have a shipping company that needs to move containers. You want to choose the origin and destination of the shipment, the type of cargo—a large bag, a sack, a barrel, and so on—and the type of container for the cargo. You also want to distribute the application load evenly, so you use Akka Sharding. To do so, add the following dependency in build.sbt:

```
val AkkaVersion = "2.6.20"
libraryDependencies += "com.typesafe.akka" %% "akka-cluster-sharding-typed"
➥% AkkaVersion
```

Let's start by defining the actor's protocol so you can add loads to a container and check its status:

```
object Container {

  final case class Cargo(id: String, kind: String, size: Int)

  sealed trait Command
  final case class AddCargo(cargo: Cargo)
      extends Command
      with CborSerializable
  final case class GetCargos(replyTo: ActorRef[List[Cargo]])
      extends Command
      with CborSerializable

  ...
}
```

To become an entity, an actor needs a key of type `EntityTypeKey` and an ID (a `String`). The combination of the two must be unique in the cluster. The `EntityTypeKey` refers to the type of the actor. This way, you can have two different types of entities with the same ID without collisions. The following listing shows the entity's ID (`containerId`) in its constructor. The container is initialized with an empty list (no cargo) and set to the `ready` behavior that allows you to `AddCargo` and `GetCargos`.

Listing 9.1 Container behavior with `containerId` and `TypeKey`

```
object Container {

  val typeKey =
    EntityTypeKey[Container.Command]("container-type-key")        ❶
```